HYDRA BTP

Audit UX des Pages + Dual View

Recherche experte: navigation, simplicite, et decisions table/cartes par page

1. Recherche: comment les grands logiciels gerent leurs pages

Apres recherche approfondie sur Procore (1.5M utilisateurs BTP), Fieldwire (Hilti), Linear, Notion, Stripe, Facebook/Instagram, et les ERP BTP (Acumatica, Sage, CMiC), voici les 5 principes universels qui emergent.

1.1 Procore: deux niveaux (Company + Project)

Procore separe strictement Company level (admin, users, config) de Project level (daily work: drawings, RFIs, financials, daily logs). Chaque niveau a ses propres "tools" dans la barre laterale. Le niveau Project est celui que 90% des utilisateurs voient quotidiennement. Lesson: HYDRA devrait faire pareil — la page d'accueil du chantier est le hub quotidien, pas la page d'admin.

1.2 Facebook/Instagram: la simplicite par reduction

Instagram a reussi la ou Facebook a echoue: simplicite radicale. Instagram a 5 onglets (Accueil, Recherche, Creer, Activite, Profil). Facebook en a 15+. Plus de tabs = plus de charge cognitive = moins d'adoption. Lesson: HYDRA doit reduire, pas ajouter. Chaque page du menu doit repondre a un besoin precis en 1 mot.

1.3 Linear: vitesse + clavier + minimalisme

Linear (500K+ utilisateurs) a bati son succes sur 3 principes: (1) interface minimaliste sans clutter, (2) tout est accessible au clavier, (3) chaque action prend < 1 seconde. Pas de sidebars multiples, pas de popups, pas de tabs cachees. Lesson: HYDRA doit reduire le nombre de clics, pas l'augmenter.

1.4 Stripe: donnees financieres = clarte par typographie

Stripe dashboard utilise la typographie (pas la couleur) pour la hierarchie. Les montants sont en gros, gras, avec tabular-nums. Les labels sont petits et gris. Le whitespace fait le travail visuel, pas les bordures. Lesson: HYDRA doit faire pareil — gros montants, petits labels, whitespace generaux.

5 principes universels retenus:
1. Default = vue dense (table) pour les listes de donnees
2. Maximum 7 items dans le menu principal
3. Une page = un objectif (pas de multi-tabs qui dispersent)
4. Gros montants, petits labels, whitespace
5. Toggle de vue persistant (localStorage)

2. Audit page par page: dual view necessaire?

Chaque page du menu HYDRA a ete analysee: type de contenu, frequence d'usage, besoin de scan rapide, et decision sur le dual view.

Page	Type	Dual View?	Raison
Accueil	Dashboard	NON	Cartes de stats + raccourcis. Pas une liste de donnees.
Mes taches	Inbox	NON (cards)	Action-oriented. Chaque carte = une decision. Table perd le contexte.
Tresorerie (hub)	Navigation	NON	3 cartes (caisses, sous-traitants, nouvelle operation). Hub, pas liste.
Espace Finance	Dashboard	NON	Cartes de caisses + pending + export. Dashboard, pas liste.
Caisses (pot list)	Liste	OUI	Liste de caisses. Table = nom, type, solde, statut. Cards actuelles OK en seconda
Caisse detail	Liste + table	OUI (fait)	Historique des operations. Tableau deja implemente (commit ebc739a).
Sous-traitants	Liste	OUI	Liste de tiers. Table = nom, type, telephone, total paye.
Operations (detail)	Fiche	NON	Detail d'une operation. Pas une liste.
Operations (form)	Formulaire	NON	Creation/modification. Pas une liste.
Validation (desk)	Decision	NON	Dossier d'approbation. Pas une liste. Do NOT table-ify.
Processus (hub)	Navigation	NON	Hub de lancement. Pas une liste.
Chantiers (list)	Liste	OUI	Liste de projets. Table = nom, type, statut, budget.
Chantier detail	Tabs	NON	Onglets (Dossiers, Equipe, Processus). Pas une liste simple.
Taches/Signalements	Liste	OUI	Liste de tickets. Table = titre, statut, assigne, date.
Documents (list)	Liste	OUI	Liste de documents. Table = reference, titre, type, statut.
Utilisateurs	Liste	OUI	Liste d'utilisateurs. Table = nom, email, role, statut.
Roles	Liste	NON	Faible volume (5-10 roles). Cards suffisent.
Journal (audit)	Liste	NON	Log chronologique. Pas besoin de toggle — table naturelle.

Resume: 6 pages sur 18 ont besoin du dual view. Les 12 autres sont des dashboards, formulaires, hubs, ou fiches detail ou le table n'apporte rien. Ne pas ajouter de toggle sur ces pages — cela ajouterait de la confusion sans valeur.

3. Ordre de priorite pour le dual view

Les 6 pages qui ont besoin du dual view, dans l'ordre d'impact:

#	Page	Key	Default	Colonnes table	Impact
1	Caisse detail (historique)	movements	table	Date, Libelle, Tiers, Type, Mode, Montant, Statut	FAIT (commit ebc739a)
2	Sous-traitants	parties	table	Nom, Type, Telephone, Total paye, Dernier paiement	HAUT — liste longue
3	Chantiers	projects	table	Nom, Type, Statut, Solde caisse, Membres	HAUT — vue d'ensemble
4	Taches/Signalements	tickets	table	Titre, Statut, Assigne, Projet, Date	MOYEN — volume modere
5	Documents	documents	table	Reference, Titre, Type, Dossier, Statut	MOYEN — volume modere
6	Utilisateurs	users	table	Nom, Email, Role, Actif	BAS — faible volume

4. Ce qu'on change et pourquoi

4.1 Caisses (pot list) — ajouter dual view

Actuellement: cartes uniquement. Pourquoi table: un gérant avec 9 caisses (1 centrale + 8 chantiers) veut comparer les soldes en un coup d'œil. Une table triée par solde permet de voir quelle caisse est presque à sec. Les cartes nécessitent de scroller.

4.2 Sous-traitants (party list) — ajouter dual view

Actuellement: cartes uniquement. Pourquoi table: 10+ sous-traitants, on veut voir le total payé à chacun trié par montant. Une table avec colonne "Total payé" permet de comparer. Les cartes montrent le total mais pas côte à côte.

4.3 Chantiers (projects list) — ajouter dual view

Actuellement: grille de cartes. Pourquoi table: 9 projets, on veut comparer statut + solde caisse + nombre de membres. Table = vue comparative. Cards = vue visuelle par projet.

4.4 Tâches/Signalements — ajouter dual view

Actuellement: liste de cartes. Pourquoi table: les tickets sont des items structurés (titre, statut, assigné, date). Une table permet de trier par statut ou assigné. Cards restent utiles pour le scan visuel.

4.5 Documents — ajouter dual view

Actuellement: grille/grille. Pourquoi table: les documents ont une reference, un type, un dossier — des champs naturels pour une table. Cards restent pour la vue visuelle (aperçu).

4.6 Utilisateurs — ajouter dual view

Actuellement: liste. Pourquoi table: les utilisateurs ont email + role + statut qui sont parfaits pour une table. Faible volume (5-15) donc cards OK aussi.

4.7 Pages qui NE changent PAS

Les pages suivantes ne reçoivent PAS de dual view, avec raison:

Page	Raison de ne PAS changer
Accueil	Dashboard de stats. Pas une liste de donnees.
Mes taches	Action-oriented. Table perd le contexte de decision.
Tresorerie hub	Navigation. 3 cartes. Pas une liste.
Espace Finance	Dashboard. Cartes de caisses + pending.
Operations detail	Fiche detail. Pas une liste.
Operations form	Formulaire. Pas une liste.
Validation desk	Dossier d'approbation. Do NOT table-ify.
Processus hub	Navigation + lancement.
Chantier detail	Tabs (Dossiers, Equipe, Processus). Pas une liste simple.
Roles	5-10 items. Cards suffisent.
Journal	Log chronologique. Table naturelle, pas besoin de toggle.

5. Conclusion

Le dual view (table + cards) est un pattern puissant pour les pages de liste où l'utilisateur scanne, compare, et trie. Mais appliqué aveuglément à toutes les pages, il devient du bruit. La règle: **si la page affiche une liste de données structurées (plus de 5 items) que l'utilisateur trie ou compare → table.** Si la page est un dashboard, un hub, un formulaire, ou une fiche detail → pas de toggle, pas de table.

> Le travail restant: appliquer le pattern ViewModeService aux 5 pages identifiées (parties, projects, tickets, documents, users). Le pattern est identique à celui déjà implémenté sur le pot-detail (commit ebc739a): ViewModeService.forPage(key) + toggle + table + cards. Chaque page prend 30 minutes.